RAG Fixes: Intent Filtering and Source Checking

Project: eCOM-AI | Area: AI chat retrieval safety and relevance

> **Short version:** The app had two common RAG problems: it could retrieve the wrong type of source, and it could answer from sources that were related but did not actually support the question. We fixed this with intent filtering and source coverage checking.

1. Issue A: Intent / Source-Type Filtering

Some questions should search products, while others should search policy or delivery documents.

Example Before Fix

```
Question:
What products are available under 20 dollars?

Bad behavior:
The system returned delivery options, especially FREE delivery.
```

Why this happened:

- The phrase `under 20 dollars` matched cheap delivery documents.
- Vector search knew the words were related to price, but it did not understand the user asked for products.
- The app needed to restrict the search to `sourceType = product`.

Remedy Taken

We added intent detection in `Infrastructure/Services/AiChatService.cs`.

The app now detects product language and delivery language before retrieval.

```
private static string DetectPreferredSourceType(string question)
{
    var normalizedQuestion = question.ToLowerInvariant();

    if (ContainsAny(normalizedQuestion, "delivery", "shipping", "ship", "shipped"))
    {
        return PolicySourceType;
    }

    if (ContainsAny(
        normalizedQuestion,
        "product",
        "products",
        "hat",
        "hats",
        "glove",
        "gloves",
        "boot",
        "boots",
```

```
        "board",
        "boards",
        "react",
        "netcore",
        "angular",
        "under",
        "cheapest",
        "expensive"))
    {
        return ProductSourceType;
    }

    return null;
}
```

That detected source type is passed to the retriever:

```
_aiRetrieverService.SearchAsync(question, 5, preferredSourceType, maximumPrice)
```

The retriever then filters candidate matches by `sourceType`:

```
if (!string.IsNullOrWhiteSpace(preferredSourceType))
{
    candidateMatches = candidateMatches.Where(match =>
        string.Equals(match.Chunk.SourceType, preferredSourceType, StringComparison.OrdinalIgnoreCase));
}
```

Result After Fix

Question	Expected Source Type	After Fix
What products are available under 20 dollars?	product	**Returns products under $20, not delivery options.**
What delivery options are available?	policy	**Returns delivery policy documents.**
Show me hats	product	**Returns hat products.**

2. Issue B: Price Filtering

For questions with a maximum price, retrieval alone is not enough. The app must use the real numeric price field.

Example

```
Question:
What products are available under 20 dollars?
```

Intent detection says this is a product question. Price extraction then detects the maximum price:

```
private static double? DetectMaximumPrice(string question)
{
    var match = Regex.Match(
        question,
        @"\b(?:under|below|less than|cheaper than)\s*\$?\s*(\d+(?:\.\d+)?)",
        RegexOptions.IgnoreCase);

    if (!match.Success)
    {
        return null;
    }

    if (double.TryParse(match.Groups[1].Value, out var maximumPrice))
    {
        return maximumPrice;
    }

    return null;
}
```

The retriever uses that value to keep only documents with a real price below the number:

```
if (maximumPrice.HasValue)
{
    candidateMatches = candidateMatches.Where(match =>
        TryGetMetadataDouble(match.Chunk, "price", out var price)
        && price < maximumPrice.Value);
}
```

3. Issue C: Source Coverage Checking

This is the safety check that prevents the app from answering from weak or unrelated sources.

Example Before Fix

```
Question:
Do you offer cash on delivery?

Bad behavior:
The retriever found delivery documents and answered with delivery options.
```

Why this was wrong:

- The word `delivery` matched delivery documents.
- But those documents did not mention `cash`, `COD`, or payment methods.
- A related source is not always an answering source.

Remedy Taken

We added `HasSourceCoverage`. It checks whether the retrieved source text actually contains the key concept needed to answer the question.

```
if (ContainsAny(normalizedQuestion, "cash on delivery", "cod"))
{
    return ContainsAny(sourceText, "cash on delivery", "cod", "cash", "payment");
}
```

More examples of coverage checks:

```
discount question  -> source must mention discount, coupon, sale, or promotion
next day question  -> source must mention next day or overnight
international      -> source must mention international, worldwide, countries, or country
return/refund      -> source must mention return or refund
warranty           -> source must mention warranty or guarantee
crypto             -> source must mention crypto, cryptocurrency, or bitcoin
store locations    -> source must mention store, location, or New York
```

Result After Fix

Question	Why It Should Not Answer	After Fix
Do you offer cash on delivery?	Delivery docs do not confirm cash/COD payment.	**Rejects safely.**
Do you have discounts on hats?	Hat products exist, but no discount source exists.	**Rejects safely.**
Can I get next day delivery?	Delivery docs do not confirm next-day/overnight delivery.	**Rejects safely.**
What warranty comes with laptops?	No warranty/laptop policy is present in retrieved sources.	**Rejects safely.**

4. How The Fixed Flow Works

```
User question
  -> Detect source intent: product or policy
  -> Detect structured filters, such as maximum price
  -> Retrieve relevant documents
  -> Apply source-type and price filters
  -> Check confidence threshold
  -> Check source coverage
  -> Build answer with sources
```

5. Why This Matters Before Adding an LLM

An LLM can write better answers, but it should not be trusted to guess policies or prices.

These fixes make the RAG pipeline safer before LLM generation:

- **Intent filtering** keeps product questions away from policy documents and policy questions away from products.
- **Price filtering** uses real structured prices instead of language similarity.
- **Source coverage checking** stops the app from answering unsupported questions.

6. Remaining Nice-To-Have

The final answer is still template-based. The app now retrieves and validates sources better, but an LLM can later turn verified sources into more natural answers.

Recommended LLM role later:

- Use only the retrieved and validated sources.
- Write a clean customer-facing answer.
- Cite the product or policy sources.
- Say it cannot confirm something when the source coverage check fails.